

실시간 라이브 스트리밍 데이터 분석 플랫폼

프로젝트 기획 및 개발 명세서

물리적 CQRS 아키텍처 · Kafka 이벤트 스트리밍 · InfluxDB 시계열 분석 · FinOps 기반 GKE 인프라

V1.2.0 · 2026-07-03 · 공동 개발 TF (2인)

목차

1. 문서 개요 및 관리 이력
2. 프로젝트 개요
3. 전체 아키텍처 개요
4. 데이터 흐름 상세
5. 구조 설계 (물리적 CQRS 및 분산 정합성)
6. 백엔드 및 데이터 파이프라인 상세 설계
7. 데이터 스토리지 명세
8. 프론트엔드 대시보드 및 서빙 API 명세
9. 인프라 및 가용성 · SRE 상세 설계
10. 조직 내 역할 분담 (R&R)
11. 기술 스택 및 구현 대상 매핑
12. 개발 단계별 로드맵

1. 문서 개요 및 관리 이력

관리 항목	내용
프로젝트명	실시간 대용량 스트리밍 피드백 분석 플랫폼
문서 버전	V1.2.0 (InfluxDB 시계열 스토리지 반영본)
최종 수정일	2026-07-03
작성자	진(Jin) 및 공동 개발 TF (총 2인 — 프론트+백엔드 / 인프라)
문서 목적	팀 내부 기술 합의, 신규 합류자 온보딩, 구현 단계 참조용 가이드

2. 프로젝트 개요

2.1 배경 및 목적

치지직(Chzzk), 유튜브 라이브(YouTube Live)와 같은 고부하 실시간 스트리밍 환경에서는 채팅 트래픽, 초 단위 시청자 수, 후원(도네이션) 등 다차원 데이터가 초당 수만 건씩 발생한다. 이 데이터를 유실 없이 수집하고 시계열로 분석할 수 있는 플랫폼을 구축하는 것이 본 프로젝트의 목적이다.

기존의 단일 CRUD 구조는 쓰기(등록·수정)와 읽기(조회) 트래픽이 같은 저장소에서 경합하기 때문에, 트래픽이 몰리는 순간 조회 성능이 함께 저하되는 한계가 있다. 이를 극복하기 위해 쓰기 저장소와 읽기 저장소를 물리적으로 분리하는 CQRS(Command Query Responsibility Segregation) 패턴과, 서비스 간 상태 변화를 이벤트로 전파하는 이벤트 기반 아키텍처(EDA, Event-Driven Architecture)를 채택한다.

이 구조를 통해 데이터 처리의 멍등성(Idempotency), 정확히 한 번 처리(Exactly-Once Semantics), 서비스 메시 기반 보안(mTLS)이라는 세 가지 과제를 인프라와 애플리케이션 레벨에서 함께 검증하는 것을 팀의 학습 목표로 삼는다.

2.2 핵심 설계 용어

아래 다섯 개 용어는 문서 전반에서 반복적으로 등장하므로, 처음 문서를 접하는 팀원을 위해 먼저 정의한다.

용어	정의
CQRS	명령(쓰기)과 조회(읽기)의 책임과 저장소를 분리하는 설계 패턴. 본 프로젝트에서는 MySQL/InfluxDB(쓰기 계열)와 Redis(읽기 계열)로 물리적으로 분리한다.

용어	정의
EDA	서비스 간 상태 변화를 직접 호출 대신 이벤트(메시지)로 전파하는 아키텍처. Kafka가 이벤트 버스 역할을 한다.
멱등성(Idempotency)	동일한 요청이 여러 번 처리되어도 결과가 달라지지 않는 성질. 후원 결제처 럼 중복 처리가 사고로 이어지는 도메인에 필수.
Exactly-Once	메시지가 중복되거나 유실되지 않고 정확히 한 번만 처리되는 것을 보장하는 수준. Kafka의 트랜잭션 기능으로 구현.
mTLS	통신 양쪽(클라이언트·서버)이 서로의 인증서를 검증하는 상호 TLS. 서비스 메 시(Istio) 레벨에서 강제한다.

2.3 목표 성능 지표 (Target SLA)

지표	목표치	비고
수집 처리량 (Ingestion Throughput)	≥ 50,000 msg/s	동시 채팅 패킷 드롭 없이 수집
End-to-End 레이턴시	≤ 1.5초	외부 API 수집 시점 ~ 대시보드 WebSocket 푸시
정산 데이터 정확도	중복·유실률 0%	후원(Superchat) 및 정산 도메인, Exactly-Once로 보 장
장애 자동 복구 시간	≤ 30초	GKE 노드 또는 Kafka 브로커 다운 시 Failover
인프라 비용	GCP \$300 크레딧 내	상시 가동 + 부하 테스트 가능한 구조, FinOps 최적 화

3. 전체 아키텍처 개요

아래는 물리적 CQRS 구조를 기준으로 그린 전체 시스템 아키텍처다. 주황색 영역이 쓰기(Command) 경로, 초록색 영역이 읽기(Query) 경로다. 두 경로는 Kafka를 경계로 물리적으로 분리되어 있어, 쓰기 트래픽이 물려도 조회 성능에 영향을 주지 않는다.

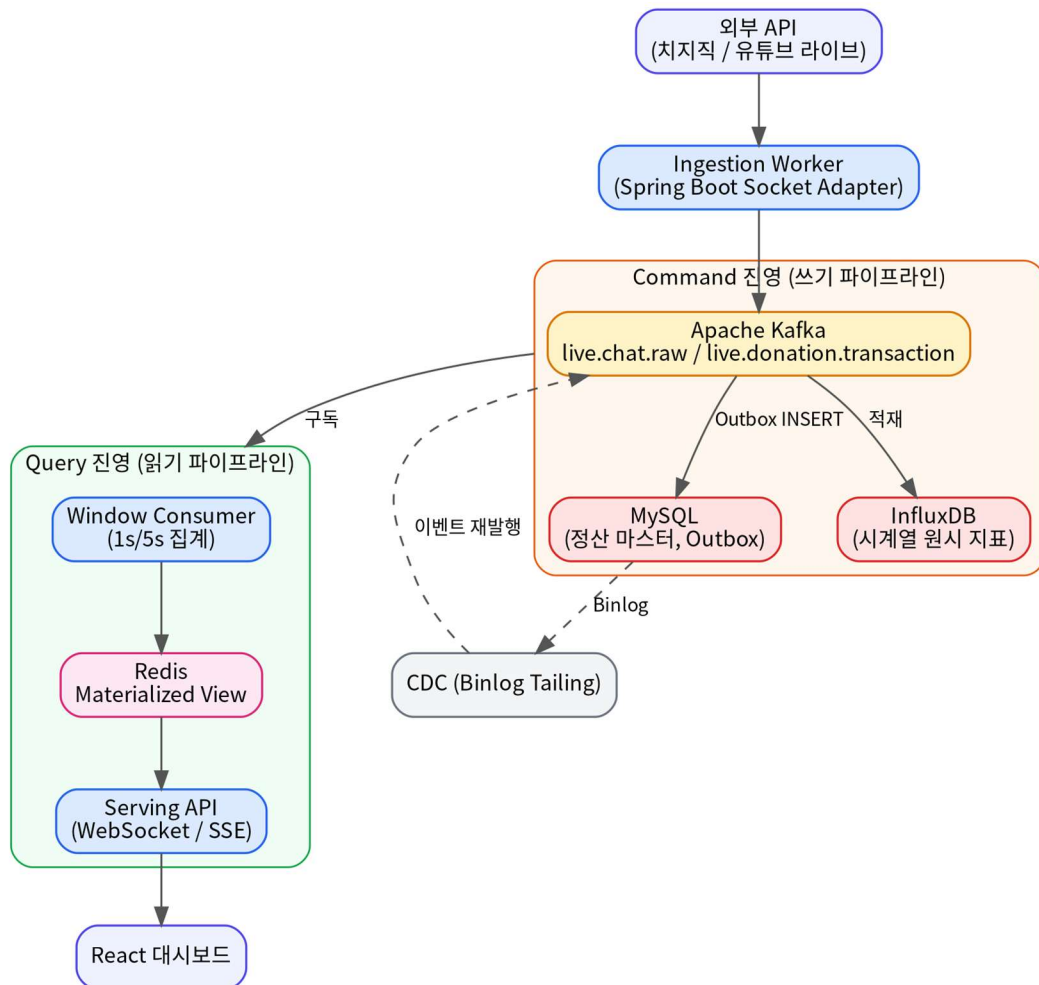


그림 1. 물리적 CQRS 아키텍처 — Command 진영과 Query 진영은 Kafka를 경계로 분리된다

3.1 구성 요소 요약

구성 요소	역할
Ingestion Worker	외부 스트리밍 API(치지직·유튜브)와 소켓/폴링으로 연결해 원시 데이터를 수집하는 Spring Boot 애플리케이션
Apache Kafka	수집된 데이터를 안전하게 버퍼링하고 여러 소비자에게 전달하는 분산 메시지 브로커. Command/Query 진영을 잇는 경계
MySQL	정산 및 마스터 데이터를 저장하는 관계형 DB. Transactional Outbox의 발행 지점이기도 하다

구성 요소	역할
InfluxDB	채팅·시청자 수 등 대용량 시계열 원시 지표를 저장하는 시계열 전용 DB
Window Consumer	Kafka 이벤트를 구독해 1초/5초 단위로 집계 연산을 수행하는 스트림 처리 컴포넌트
Redis	집계 결과를 구체화된 뷰(Materialized View)로 저장해 저지연 조회를 제공하는 인메모리 스토어
Serving API	Redis/InfluxDB의 데이터를 WebSocket-SSE로 프론트엔드에 실시간 전달하는 게이트웨이
React 대시보드	Zustand로 상태를 관리하며 실시간 지표를 시각화하는 프론트엔드

4. 데이터 흐름 상세

이 장은 데이터 한 건이 외부 API에서 발생해 사용자 화면에 표시되기까지의 전체 여정을 단계별로 설명한다. 처음 프로젝트에 합류하는 팀원은 이 장만 읽어도 시스템의 데이터 흐름을 이해할 수 있도록 작성했다.

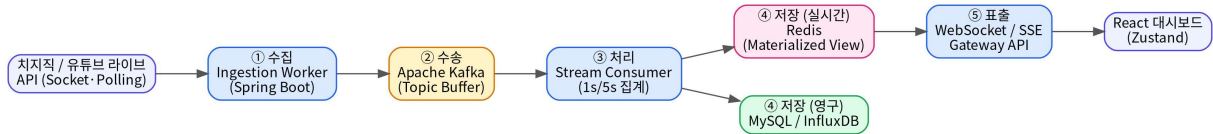


그림 2. 데이터 파이프라인 5단계 — 수집 → 수송 → 처리 → 저장 → 표출

4.1 파이프라인 5 단계

단계	구성 요소	설명
① 수집 (Ingestion)	Ingestion Worker	치지직·유튜브 API에 소켓/폴링 연결을 수립해 채팅, 시청자 수, 후원 이벤트를 실시간으로 획득한다.
② 수송 (Transport)	Apache Kafka	수집된 원시 데이터를 유실 없이 Kafka 토픽 버퍼로 전송한다. 대량 트래픽의 완충 지대 역할을 한다.
③ 처리 (Processing)	Window Consumer	Kafka 이벤트를 구독해 1초/5초 슬라이딩 윈도우 단위로 채팅 밀도, 시청자 증감 속도 등을 실시간 연산한다.
④ 저장 (Storage)	Redis / MySQL / InfluxDB	실시간 서빙용 지표는 Redis에 캐싱하고, 영구 보관이 필요한 통계·정산 데이터는 MySQL과 InfluxDB에 각각 적재한다.
⑤ 표출 (Serving)	WebSocket / SSE / React	가공된 지표를 WebSocket·SSE로 1초 주기 푸시하며, 프론트엔드는 필요한 영역만 리렌더링한다.

4.2 단계별 데이터 형태 예시

단계마다 데이터가 어떤 형태로 변환되는지 실제 예시로 살펴본다.

① 수집 단계 — 외부 API 원시 데이터

```

{
  "userId": "u_88213",
  "streamerId": "chzzk_woowakgood",
  "chatMessage": "가즈아!!",
  "timestamp": "2026-07-03T14:22:01.183Z",
  "viewerCount": 48213,
  "likeCount": 1023,
  "donationAmount": 5000,
  "transactionUuid": "a1b2c3d4-...-e5f6"
}
  
```

② 수송 단계 — Kafka 메시지 (live.donation.transaction)

```
Topic: live.donation.transaction
Key (Partition Key): streamerId = "chzzk_woowakgood"
Headers: { eventType: "DONATION_CREATED", traceId: "..." }
Value: 위 원시 데이터 JSON (Outbox를 통해 발행)
```

③ 처리 단계 — 5 초 윈도우 집계 결과

```
{
  "streamerId": "chzzk_woowakgood",
  "windowStart": "2026-07-03T14:22:00Z",
  "chatCountPerSec": 812,
  "viewerDelta": +134,
  "heatScore": 0.87
}
```

④ 저장 단계 — Redis Hash / InfluxDB Line Protocol

```
# Redis: streamer:stats:chzzk_woowakgood (Hash)
viewerCount=48213 chatPerSec=812 heatScore=0.87

# InfluxDB Line Protocol
stream_metrics,streamer_id=chzzk_woowakgood viewer_count=48213i 1751551321183000000
```

⑤ 표출 단계 — WebSocket 푸시 페이로드

```
{
  "type": "METRIC_UPDATE",
  "streamerId": "chzzk_woowakgood",
  "viewerCount": 48213,
  "heatScore": 0.87,
  "pushedAt": "2026-07-03T14:22:01.980Z"
}
```

5. 구조 설계 (물리적 CQRS 및 분산 정합성)

5.1 Command / Query 인프라 격리

Command 진영(쓰기 파이프라인)은 외부 소켓 어댑터로 유입된 데이터를 Kafka 버퍼로 전송한 뒤, 정산용 관계형 DB(MySQL)와 대용량 시계열 분석 전용 DB(InfluxDB)로 이원화하여 적재한다.

Query 진영(읽기 파이프라인)은 Kafka 뒤에 붙은 Consumer가 윈도우 연산 결과만 Redis에 구체화된 뷰(Materialized View) 형태로 반영한다. Serving API는 오직 Redis와 InfluxDB의 조회용 뷰 모델에만 접근하므로, 쓰기 트랜잭션과 락 경합을 원천적으로 겪지 않는다.

5.2 Transactional Outbox 패턴

로컬 DB에 비즈니스 데이터를 저장하는 것과 Kafka에 이벤트를 발행하는 것은 원래 서로 다른 두 시스템에 걸친 작업이라 원자성을 보장하기 어렵다. 분산 트랜잭션(2PC) 대신 Transactional Outbox 패턴을 도입해 이 문제를 해결한다.

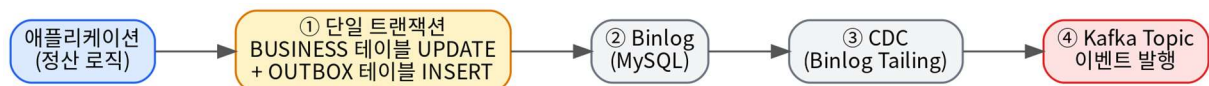


그림 3. Transactional Outbox 패턴 — DB 트랜잭션과 이벤트 발행의 원자성을 보장한다

단계	설명
① 단일 트랜잭션	비즈니스 테이블 UPDATE와 OUTBOX 테이블 INSERT를 같은 DB 트랜잭션 안에서 함께 커밋한다.
② Binlog 기록	MySQL이 커밋된 변경 사항을 바이너리 로그(Binlog)에 순서대로 기록한다.
③ CDC 캡처	CDC(Change Data Capture) 툴이 Binlog를 실시간으로 읽어 변경분을 감지한다.
④ Kafka 이벤트 발행	CDC가 감지한 OUTBOX 레코드를 Kafka 토픽으로 안전하게 재발행한다.

이 구조로 Write DB(MySQL)와 Read DB(Redis/InfluxDB) 사이에는 짧은 복제 지연(Replication Lag)이 존재하지만, 그 구간을 결과적 정합성(Eventual Consistency)으로 명시적으로 관리한다.

6. 백엔드 및 데이터 파이프라인 상세 설계

6.1 Kafka 토픽 설계

토픽명	파티션	처리 보증 수준	Partition Key
live.chat.raw	3개 이상	At-least-once (성능 지향)	streamer_id
live.donation.transaction	-	Exactly-Once (EOS v2)	streamer_id

live.donation.transaction 토픽은 금융 정산이 걸려 있으므로 아래 설정으로 Exactly-Once를 강제한다.

```
# Producer
enable.idempotence = true
acks = all
max.in.flight.requests.per.connection = 1

# Consumer / Broker
processing.guarantee = exactly_once_v2
# __transaction_state 토픽 + Transaction Coordinator로 2PC 수행
```

6.2 동시성 제어 전략

문제 상황	해결 기법	적용 위치
다수 컨슈머 락이 동일 스트리머 통계를 동시 갱신 (Race Condition)	Redisson Pub/Sub 기반 분산 락	Redis
결제 트래픽 폭주 구간의 잔액 차감 데드락	JPA 낙관적 락 (@Version) + 비관적 락(SELECT FOR UPDATE) 이원화	MySQL
동일 방송방 이벤트의 순서 꼬임	Kafka Partition Key = streamer_id 고정	Kafka
결제 로직 예외로 인한 전체 파이프라인 정지 (Head-of-Line Blocking)	DLQ(Dead Letter Queue)로 에러 메시지만 우회	Kafka Consumer

낙관적 락은 자원 경쟁 실패 시 Kafka 재시도 토픽으로 라우팅하고, 비관적 락은 계좌 로우를 직접 잠가 데드락 가능성이 높은 정산 도메인에 한정해 적용한다. 두 방식을 도메인 민감도에 따라 나누어 튜닝 테스트를 진행한다.

7. 데이터 스토리지 명세

7.1 MySQL — 정산 및 마스터 데이터

컬럼명	타입	제약조건	설명
account_id	VARCHAR(64)	PRIMARY KEY	스트리머·유저 계좌 고유 식별자
current_balance	BIGINT	NOT NULL DEFAULT 0	현재 정산 가능한 포인트/통화 잔액
version	BIGINT	NOT NULL DEFAULT 0	JPA 낙관적 락 구현용 필드

7.2 InfluxDB — 시계열 데이터 (Bucket: live-stream-analytics)

구분	이름	타입	설명
Measurement	stream_metrics	-	시계열 데이터 최상위 집합 (RDB의 테이블에 대응)
Tag (Indexed)	streamer_id	String	방송 채널 식별 키. 인덱싱되어 고속 쿼리 지원
Field	chat_message	String	실시간 원시 채팅 문자열 (인덱스 없음)
Field	viewer_count	Integer	초 단위 실시간 시청자 수 변동 메트릭

7.3 Redis — 실시간 서빙 및 캐시

Key 구조	타입	용도 및 TTL
streamer:stats:<streamer_id>	Hash	현재 시청자 수·초당 채팅 유입 수 서빙용 (TTL 없음)
lock:streamer:<streamer_id>	String (NX)	Redisson 분산 락 획득 제어용 (Lease Time: 2초)

8. 프론트엔드 대시보드 및 서버 API 명세

8.1 기술 스택

항목	선택
프레임워크	TypeScript + React / Next.js
전역 상태 관리	Zustand — 빈번한 실시간 갱신에 적합한 가벼운 상태 매니저
렌더링 최적화	실시간 데이터 수신 시 컴포넌트 전체가 아닌 캔버스·텍스트 노드 단위로만 부분 업데이트

8.2 API 엔드포인트

프로토콜	경로	설명
SSE (단방향)	GET /api/v1/streamer/{streamerId}/metrics/live	시청자 수 추이 등 실시간 지표를 서버→클라이언트로 단방향 스트리밍
WebSocket (양방향)	/ws/v1/chat/feed	실시간 하이라이트 채팅과 고액 후원 알림 트리거를 전송하는 양방향 채널

9. 인프라 및 가용성 · SRE 상세 설계

우선순위 표기 안내

- MVP (초록): 핵심 파이프라인 완성을 위해 반드시 먼저 구현하는 항목
- 스트레치 (주황): MVP 완성 후 단계적으로 붙여가며 정량 지표(before/after)를 확보하는 확장 검증 항목

9.1 GKE 클러스터 구축 및 FinOps

- **MVP** 인프라 형상 관리: VPC, Subnet, GKE Cluster 등 모든 자원을 Terraform 으로 100% 코드화(IaC).
- **MVP** 클러스터 아키텍처: GCP 비용 절감을 위해 단일 영역(Zonal Single Cluster)으로 구성.
- **MVP** Spot VM 노드풀: On-demand 대비 최대 90% 저렴한 Spot VM 사용. PodDisruptionBudget 과 30 초 이상의 Termination Grace Period 로 갑작스러운 회수(Eviction) 시에도 무중단 마이그레이션 보장.
- **MVP** 야간 자동 셧다운: 사용량이 없는 00:00~07:00(KST)에는 CronJob 이 gcloud container clusters resize 를 실행해 노드를 0 개로 다운스케일링.
- **스트레치** 비용 시각화(Kubecost): GCP 실제 빌링 API 와 연동해 Spot VM·KEDA 적용 전후의 절감액을 달러 단위로 대시보드화. MVP 완성 후 붙여서 "적용 전/후" 비교 수치를 포트폴리오 지표로 확보한다.

9.2 자동 확장 및 네트워킹

- **MVP** KEDA (이벤트 기반 오토스케일링): Kafka Consumer Lag(records-lag) 지표를 직접 감시하는 ScaledObject 로 트래픽 폭주 시 컨슈머 팟을 스케일 아웃한다. maxReplicaCount 는 Kafka 토픽의 파티션 수와 동일하게 제한해 불필요한 팟 생성을 차단한다.
- **MVP** Istio Service Mesh: 정산·계좌 관련 네임스페이스에 PeerAuthentication 을 Strict 모드로 고정해 mTLS 를 강제하고, 서킷 브레이커로 외부 API 장애 전파를 차단한다.
- **스트레치** Cilium (eBPF CNI): kube-proxy 의 iptables O(N) 순차 탐색 대신, 커널 레벨 eBPF 로 O(1) 해시 맵 라우팅을 수행해 네트워킹 지연을 최소화한다. MVP 가 Istio 기반으로 안정화된 뒤, 같은 클러스터에 붙여서 iptables 대비 레이턴시 개선폭을 정량 비교한다.

9.3 보안 거버넌스

- **MVP** External Secrets Operator(ESO)를 도입해 DB·Kafka 비밀번호 같은 민감 정보를 GCP Secret Manager 에 보관하고, K8s 클러스터에는 암호화된 토큰으로 동적 주입한다. 이는 AWS 의 IRSA 아키텍처와 유사한 개념으로, GCP Workload Identity 와의 호환성을 함께 검증한다.

9.4 모니터링 및 카오스 엔지니어링

- MVP** Prometheus / Alertmanager: Kafka Consumer Lag 이 5,000 개 이상으로 3 분 이상 지속되거나 GKE 노드가 다운되면 Slack 웹훅으로 온콜 경보를 전송.
- 스트레치** Chaos Mesh: 부하 테스트 중 의도적으로 Kafka 브로커 팻을 강제 종료하거나 500ms 네트워크 지연을 주입해, KEDA 와 Istio 가 다운타임 없이 방어하는지 정량 지표로 검증하고 문서화. MVP 의 정상 동작이 먼저 검증된 뒤 마지막 단계에서 붙인다.

9.5 스트레치 목표 정리

아래 3개 항목은 핵심 파이프라인(MVP)이 완성되고 정상 동작이 확인된 뒤, 하나씩 순서대로 붙여가며 각각의 적용 전/후 정량 지표를 확보하는 방식으로 진행한다. 순서를 두는 이유는 한 번에 여러 변수를 추가하면 문제 발생 시 원인 분리가 어렵기 때문이다.

항목	확보할 정량 지표	적용 순서
Kubecost	Spot VM·KEDA 적용 전/후 월 비용 절감액 (달러)	1순위 — 설치만으로 측정 가능, 리스크 낮음
Cilium eBPF	iptables 대비 패킷 처리 지연(ms) 개선폭	2순위 — 네트워킹 레이어 교체, 기존 Istio와 호환성 확인 필요
Chaos Mesh	브로커 강제 종료 후 자동 복구 시간(초), 다운타임 여부	3순위 — 파괴 테스트이므로 안정화 이후 마지막에 수행

10. 조직 내 역할 분담 (R&R)

2인 체제(프론트+백엔드 / 인프라)로 진행하며, 경계 원칙은 다음과 같다: 데이터가 Kafka에 들어온 이후 애플리케이션 레벨에서 처리·저장되는 로직(컨슈머, 동시성 제어, API)은 백엔드 담당, 그 로직이 올라가 도는 플랫폼 자체(클러스터·네트워크·브로커 운영·관측)는 인프라 담당으로 나눈다.

담당 파트	담당자	상세 업무 범위
프론트엔드 & 백엔드	파트너	치지직/유튜브 소켓 어댑터 수집기 개발, TypeScript+React-Zustand 기반 실시간 차트 UI 구현, WebSocket-SSE 게이트웨이 서버 구축, Kafka Consumer 가공 로직(1s/5s 윈도우 집계) 구현, Redisson 분산 락 및 JPA 낙관/ 비관적 락을 활용한 동시성 제어, Redis-MySQL-InfluxDB 연동 API 구현
인프라 & 플랫폼	진 (Jin, 본인)	Terraform 기반 GCP 인프라 자동화, GKE-Spot VM 노드풀 최적화, 야간 자동 셧다운 스크립트, Kafka 클러스터 구축 및 브로커 레벨 EOS 설정(파티션·트랜잭션 코디네이터), KEDA 스케일러 연동, Istio mTLS 보안 인프라 배포, Prometheus/Alertmanager 관측망 구축, [스트레치] Kubecost-Cilium eBPF-Chaos Mesh 단계적 적용
공동 도메인 연동	공동 작업	Kafka 메시지 JSON 스키마 합의, Redis Key-Value 구조 및 서빙 포맷 확정, End-to-End 통합 부하 테스트 및 병목 트러블슈팅

11. 기술 스택 및 구현 대상 매핑

기술 스택	레이어	구현 목적
GKE (K8s)	인프라 오케스트레이션	전체 마이크로서비스 컨테이너 구동, 무중단 배포 및 자원 격리
Terraform	인프라 형상 자동화 (IaC)	명령 한 번으로 GCP 네트워크·클러스터를 생성·파괴하는 비용 절감형 스크립트 자산화
Apache Kafka	대량 데이터 수송	초당 수만 건 채팅 트래픽 버퍼링, 정산 트랜잭션 Exactly-Once 보장
KEDA	이벤트 기반 자동 확장	Consumer Lag 지표 기반 컨슈머 팟 자동 스케일 아웃
Istio Mesh	서비스 메시 보안	정산 네임스페이스 mTLS Strict 강제, 서킷 브레이커
Cilium (eBPF) [스트레치]	고속 커널 네트워킹 CNI	iptables 우회, O(1) 해시맵 라우팅으로 지연 최소화. MVP 안정화 후 적용
Spring Boot	백엔드 수집·서빙	외부 소켓 어댑터 통신, API 게이트웨이 구동
Redisson (Redis)	분산 동시성 제어	멀티 팟 환경에서 동일 스트리머 통계 갱신 시 Pub/Sub 분산 락
MySQL & JPA	정산 마스터 스토리지	ACID 금융 원장, 낙관적/비관적 락 구현
InfluxDB	시계열 데이터 스토리지	대규모 채팅·시청자 지표를 물리 격리된 고성능 환경에 적재
React & Zustand	프론트엔드 대시보드	1초 주기 실시간 차트·후원 알림, 가상 돔 부하 최소화
Prometheus & Alertmanager	SRE 모니터링	Consumer Lag·노드 다운 시 Slack 온콜 경보
Chaos Mesh [스트레치]	복구력 검증	브로커 강제 종료·네트워크 지연 주입으로 다운타임 제로 검증. MVP 완성 후 마지막에 적용
Kubecost [스트레치]	비용 관측(FinOps)	GCP 실빌링 연동, Spot VM·KEDA 적용 전/후 절감액을 달리로 시각화. MVP 완성 후 가장 먼저 적용

12. 개발 단계별 로드맵

12.1 MVP 단계 (Phase 1~4)

핵심 파이프라인을 완성하는 단계. 이 4단계가 끝나야 "동작하는 CQRS 스트리밍 플랫폼"이 성립한다.

단계	주요 목표	핵심 작업
Phase 1 인프라 뼈대 & 수집 연동	기본 파이프라인 연결	Terraform GCP 구축 → 기본 노드풀 세팅 → Kafka 단일 브로커(KRaft) 배포 → Spring Boot 수집 어댑터 연동 및 Raw 토픽 적재 검증
Phase 2 동시성 제어 & 물리 CQRS	CQRS 구조 완성	MySQL 정산 도메인 설계 → Redisson-JPA 락 튜닝 → 컨슈머 가공 로직 구현 → Redis/InfluxDB 프로젝트선 파이프라인 → React 대시보드 WebSocket/SSE 연동
Phase 3 가용성 기반 & FinOps	비용·가용성 최적화	GKE Spot VM 이주 및 야간 셧다운 크론잡 → KEDA Lag 기반 자동 확장 검증 → Istio mTLS 적용
Phase 4 SRE 관측 & 마무리	운영 신뢰성 확보	Prometheus/Alertmanager 온콜 연동 → 전체 파이프라인 E2E 부하 테스트 → MVP 트러블슈팅 보고서 작성

12.2 스트레치 단계 (Phase 5) — MVP 완성 후 순차 적용

MVP가 안정적으로 동작함을 확인한 뒤, 아래 3개를 하나씩 순서대로 붙이며 각 단계마다 적용 전/후 지표를 캡처해 포트폴리오 수치로 남긴다. 동시에 여러 개를 붙이지 않는 이유는 문제가 생겼을 때 원인을 명확히 분리하기 위함이다.

순서	항목	이 단계에서 뽑는 포폴 수치
5-1	Kubecost 적용	Spot VM-KEDA 도입 전/후 월 인프라 비용 절감액 (달러, %)
5-2	Cilium eBPF 전환	iptables 대비 패킷 라우팅 지연(ms) 개선폭, P99 레이턴시 변화
5-3	Chaos Mesh 장애 주입	브로커 강제 종료 후 자동 복구 시간(초), 다운타임 발생 여부